8 PUZZLE CODE

Aim:

To write a Python program to solve the **8-Puzzle problem** using the **A*** search algorithm with the Manhattan distance heuristic.

Algorithm (A* Search for 8-Puzzle):

1. **Start State**
   Represent the puzzle as a list of 9 elements (0 for the blank tile).
2. **Goal State**
   Typically: [1,2,3,4,5,6,7,8,0].
3. **Heuristic Function (h)**
   Use the Manhattan distance:
   For each tile, compute how far it is from its goal position.
4. **Cost Function (f = g + h)**
   - g = cost so far (depth/number of moves).
   - h = heuristic estimate to goal.
5. **Priority Queue**
   Use a priority queue (heapq) to always expand the state with the lowest f.
6. **State Expansion**
   - Find possible moves of the blank (up, down, left, right).
   - Generate new states.
7. **Goal Test**
   If the current state equals the goal state → return solution.
8. **Trace Path**
   Reconstruct moves to display the result.

Code:

```
import heapq

def manhattan(state, goal):
    distance = 0
    for i in range(1, 9):  # ignore the blank tile (0)
        xi, yi = divmod(state.index(i), 3)
        xg, yg = divmod(goal.index(i), 3)
        distance += abs(xi - xg) + abs(yi - yg)
    return distance

def get_neighbors(state):
    neighbors = []
    zero_index = state.index(0)
    x, y = divmod(zero_index, 3)
    moves = []
    if x > 0: moves.append((-1, 0))  # up
    if x < 2: moves.append((1, 0))   # down
    if y > 0: moves.append((0, -1))  # left
    if y < 2: moves.append((0, 1))   # right

    for dx, dy in moves:
        new_x, new_y = x + dx, y + dy
        new_index = new_x * 3 + new_y
        new_state = state[:]
```

```
        new_state[zero_index], new_state[new_index] = new_state[new_index], new_state[zero_index]
        neighbors.append(new_state)
    return neighbors

def a_star(start, goal):
    frontier = []
    heapq.heappush(frontier, (manhattan(start, goal), 0, start, []))
    explored = set()

    while frontier:
        f, g, state, path = heapq.heappop(frontier)

        if state == goal:
            return path + [state]

        explored.add(tuple(state))

        for neighbor in get_neighbors(state):
            if tuple(neighbor) not in explored:
                new_g = g + 1
                new_f = new_g + manhattan(neighbor, goal)
                heapq.heappush(frontier, (new_f, new_g, neighbor, path + [state]))

    return None
```

```
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

start_state = [1, 2, 3,
               4, 0, 6,
               7, 5, 8]
goal_state = [1, 2, 3,
              4, 5, 6,
              7, 8, 0]

solution = a_star(start_state, goal_state)

if solution:
    print("Steps to solve 8-Puzzle:")
    for step in solution:
        print_state(step)
else:
    print("No solution found.")
```

Sample input:

1 2 3

4 0 6

7 5 8

Goal state:

1 2 3

4 5 6

7 8 0

Output:

```
Enter the starting state of the 8-puzzle (use 0 for blank).
Example: 1 2 3 4 0 6 7 5 8
Start state (9 numbers separated by space): 1 2 3 4 0 6 7 5 8

Solving 8-Puzzle...

Step 0
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 1
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 2
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

Puzzle solved in 2 moves.
```

Result

The program successfully solved the 8-Puzzle problem using A* search and displayed the step-by-step states from the initial state to the goal state.